# Module 2: The Network Layer

Network layer functions and services, IPv4 addressing, subnetting and CIDR, routing algorithms, IPv4 and IPv6 datagrams, ICMP, ARP and NAT

Exam: Wednesday 18 November 2026

**Exam Focus**

**Scope.** Syllabus 2.1 to 2.4: introduction to the network layer; IP addressing, subnetting and CIDR (implemented in Cisco Packet Tracer); routing algorithms — link state, distance vector and path vector; and network layer protocols — IPv4, IPv6 and ICMP.

This module carries CO2. It is the *highest-scoring* module in the paper because it contains real calculations: subnetting, Dijkstra, Bellman-Ford and fragmentation. In the six papers, Module 2 appears in every full paper and in four of the internals.

## Contents


**1 Network layer: functions and services** 3

**2 IPv4 addressing** 3
- 2.1 Structure . . . 3
- 2.2 Classful addressing . . . 4
- 2.3 Special addresses . . . 4

**3 Subnetting** 4
- 3.1 Why subnet . . . 4
- 3.2 The subnet arithmetic . . . 4
- 3.3 CIDR . . . 5

**4 The IPv4 datagram** 6
- 4.1 Fragmentation . . . 6

**5 ICMP** 7

**6 ARP** 8

**7 IPv6** 8

**8 Routing algorithms** 9
- 8.1 The three families . . . 9
- 8.2 Link state: Dijkstra . . . 9
- 8.3 Distance vector: Bellman-Ford . . . 10
- 8.4 RIP versus OSPF . . . 10
- 8.5 Path vector and BGP . . . 11
- 8.6 README: routing tables . . . 11

**9 Answer skeletons** 12

**10 Self-test** 12

11 Last-minute page 13

# Network layer: functions and services

**Past-paper question**

**Oct 2025 Q2(b)(i).** Explain the functions of the network layer in detail. Describe the services it provides to the transport layer.

**Solution**

The network layer moves a packet from the source host to the destination host, *across multiple networks*. It is the lowest layer that deals with end-to-end addressing.

**Functions:**

1. **Logical addressing.** Assigns and uses IP addresses so that hosts on different networks can be identified uniquely. MAC addresses are only local; IP addresses are global.
2. **Routing.** Runs routing protocols to build routing tables and choose the best path from source to destination.
3. **Forwarding.** For each arriving packet, looks up the destination in the routing table and sends the packet out of the correct interface. Forwarding is per-packet and fast; routing is periodic and slow.
4. **Packetising and encapsulation.** Takes transport-layer segments and wraps them in IP packets with a header.
5. **Fragmentation and reassembly.** Splits a packet when it exceeds the MTU of a link, and reassembles at the destination (IPv4) or leaves it to the source (IPv6).
6. **Error reporting and diagnostics.** ICMP reports unreachable destinations, time exceeded, and so on.
7. **Congestion control and QoS marking.** Marks packets (DSCP) and can drop or queue them.
8. **Address translation.** NAT rewrites addresses at the boundary between private and public networks.

**Services to the transport layer:** the network layer offers a **connectionless, best-effort, unreliable** datagram service. It does not guarantee delivery, ordering, or duplicate suppression; it may drop, duplicate or reorder packets. Everything the transport layer needs on top of that — reliability, ordering, flow control — must be provided by TCP itself. This deliberate minimalism is what allows IP to run over any link technology.

| Service | Meaning |
|---|---|
| **Connectionless** | Each packet is independent; no set-up, no state kept per flow. |
| **Best effort** | The network tries but promises nothing. |
| **Unreliable** | Lost packets are not retransmitted by IP. |
| **Unordered** | Packets may arrive out of order. |

Do Not Confuse

**Routing versus forwarding.** *Routing* is the process of building the table (a control-plane activity, done by OSPF, BGP, RIP, periodically). *Forwarding* is moving one packet using the table (a data-plane activity, done per packet in nanoseconds). Exams often ask for the distinction and expect exactly this split.

# IPv4 addressing

## Structure

An IPv4 address is 32 bits, written as four decimal octets separated by dots: `192.168.10.25`. It has two parts:

$$\text{address} = \underbrace{\text{network prefix}}_{\text{identifies the network}} + \underbrace{\text{host part}}_{\text{identifies the host}}$$

The boundary between the two is given by the **prefix length** in CIDR notation, `/n`, or historically by the address

class.

### Classful addressing

| Class | First octet | Default mask | Networks | Use |
|---|---|---|---|---|
| **A** | 1–126 | /8 | 126 | Very large organisations |
| **B** | 128–191 | /16 | 16 384 | Medium organisations |
| **C** | 192–223 | /24 | 2 097 152 | Small networks |
| **D** | 224–239 | — | — | Multicast |
| **E** | 240–255 | — | — | Experimental / reserved |

### Special addresses

| Address | Meaning |
|---|---|
| `0.0.0.0` | This host, on this network; used as a source before an address is known |
| `127.0.0.0/8` | Loopback: `127.0.0.1` is the local host |
| `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16` | Private addresses (RFC 1918), not routed on the Internet |
| `169.254.0.0/16` | Link-local, self-assigned when DHCP fails |
| `255.255.255.255` | Limited broadcast |
| **Network address** | Host bits all 0; identifies the network itself |
| **Broadcast address** | Host bits all 1; reaches every host on the network |

## Subnetting

### Why subnet

**Past-paper question**

**Oct 2025 Q2(b)(iii).** What is meant by subnetting?

**Solution**

**Subnetting** is the division of one large network into smaller logical sub-networks by borrowing bits from the host part and adding them to the network part.

Reasons:

- **Reduce broadcast domains**: a broadcast reaches only its own subnet.
- **Improve security and control**: traffic between subnets can be filtered by a router or ACL.
- **Simplify administration**: each department, floor or building gets its own block.
- **Reduce wasted addresses**: a /24 gives 254 hosts; if you need 30, a /27 is enough and you can give the rest to other subnets.
- **Better performance**: less broadcast traffic and smaller routing tables through aggregation.

**The cost:** two addresses per subnet are always lost, the network address (all host bits 0) and the broadcast address (all host bits 1). So usable hosts $= 2^h - 2$, where $h$ is the number of host bits.

### The subnet arithmetic

**Remember**

**Four formulas.** For a prefix /n:

- host bits $h = 32 - n$
- addresses per subnet $= 2^h$
- usable hosts $= 2^h - 2$

- block (increment) size $= 2^h$

**The shortcut that solves most questions:** the interesting octet is the one the prefix ends in, and the subnets step by the *block size*. For `/26`, $h = 6$, block $= 64$, so the subnets in that octet start at 0, 64, 128, 192.

**Past-paper question**

**Worked, standard exam form.** Subnet `192.168.10.0/24` into 4 equal subnets. Give the network address, first host, last host and broadcast of each.

**Solution**

Four subnets need 2 borrowed bits, so the new prefix is `/26`. Then $h = 6$, block $= 2^6 = 64$, usable hosts per subnet $= 62$.

| Subnet | Network | First host | Last host | Broadcast |
|---|---|---|---|---|
| 1 | 192.168.10.0 | .1 | .62 | .63 |
| 2 | 192.168.10.64 | .65 | .126 | .127 |
| 3 | 192.168.10.128 | .129 | .190 | .191 |
| 4 | 192.168.10.192 | .193 | .254 | .255 |

The mask is `255.255.255.192`. Each subnet's broadcast is one less than the next subnet's network address — that is the check that tells you the arithmetic is right.

**Exam Focus**

**Worked, single-subnet lookup.** Given `200.1.2.0/26`, find the network, broadcast and host range.

| Item | Value |
|---|---|
| **Prefix** | /26, so $h = 6$, block $= 64$ |
| **Network address** | `200.1.2.0` |
| **Broadcast address** | `200.1.2.63` |
| **Usable host range** | `200.1.2.1` to `200.1.2.62` |
| **Usable hosts** | 62 |
| **Mask** | `255.255.255.192` |

**Exam Focus**

**Worked, the reverse direction.** A network must support 50 hosts. What prefix do you need?

**Solution**

You need $2^h - 2 \geq 50$, so $2^h \geq 52$. Since $2^5 = 32$ is too small and $2^6 = 64$ is enough, $h = 6$ and the prefix is `/26`, giving 62 usable hosts.

**Always round *up* to the next power of two.** The commonest error is choosing $h = 5$ because 50 is between 32 and 64, and forgetting the two reserved addresses.

## CIDR

**Classless Inter-Domain Routing** removes the class boundaries. Any prefix length is allowed, and addresses are written `a.b.c.d/n`. It provides:

- **Efficient allocation**: a customer needing 500 addresses gets a /23 rather than a whole class B /16.
- **Route aggregation (supernetting)**: many contiguous prefixes are advertised as one.

**Exam Focus**

**Worked, aggregation.** Summarise `192.168.0.0/24` and `192.168.1.0/24` into one prefix, and `200.1.0.0/24` through `200.1.3.0/24`.

**Solution**

**First pair:** 0 and 1 differ only in the last bit of the third octet, which becomes a host bit. The third octet had 8 network bits under /24; giving one back makes it a /23.

$$192.168.0.0/24 + 192.168.1.0/24 \;\Rightarrow\; \textbf{192.168.0.0/23}$$

**Second group:** four /24s, so two bits are given back: $24 - 2 = 22$.

$$200.1.0.0/24 \ldots 200.1.3.0/24 \;\Rightarrow\; \textbf{200.1.0.0/22}$$

**The rule:** to aggregate $2^k$ contiguous equal prefixes, subtract $k$ from the prefix length. The prefixes must be contiguous and correctly aligned; if they are not, you cannot summarise them.

## The IPv4 datagram

**Past-paper question**

**Sept 2026 CAE-1 Q5B.** Explain the structure of an IPv4 Datagram. Describe the function of each field in the IPv4 datagram header and explain how the datagram is used for delivering packets across an internetwork. Illustrate with a neat diagram.

**Solution**

The IPv4 header is 20 bytes minimum, 60 bytes maximum, and is a multiple of 4 bytes.

| Field | Size | Function |
|---|---|---|
| **Version** | 4 bits | IP version; 4 for IPv4, 6 for IPv6 |
| **Header length (IHL)** | 4 bits | Header size in 32-bit words; 5 means 20 bytes |
| **Type of service / DSCP** | 8 bits | Precedence and QoS marking; used by DiffServ |
| **Total length** | 16 bits | Header plus data, in bytes; maximum 65 535 |
| **Identification** | 16 bits | Identifies the fragments of one original datagram |
| **Flags** | 3 bits | DF (do not fragment) and MF (more fragments) |
| **Fragment offset** | 13 bits | Position of this fragment in the original, in 8-byte units |
| **Time to live (TTL)** | 8 bits | Decremented at each router; at 0 the packet is dropped and ICMP Time Exceeded is sent. Prevents loops |
| **Protocol** | 8 bits | Upper-layer protocol: 6 = TCP, 17 = UDP, 1 = ICMP, 89 = OSPF |
| **Header checksum** | 16 bits | Detects errors in the header only, recomputed at every hop |
| **Source address** | 32 bits | Sender's IP |
| **Destination address** | 32 bits | Receiver's IP |
| **Options** | variable | Rarely used: record route, timestamp, source routing |

**How it delivers a packet.** The host builds the datagram with its own address as source and the destination's as destination. Each router on the path decrements TTL, recomputes the header checksum, looks up the destination network in its routing table, and forwards the packet out of the chosen interface with a new link-layer frame (new MAC addresses, same IP addresses). IP addresses stay the same end to end; MAC addresses change at every hop. That is the single most important sentence in this topic.

### Fragmentation

If a datagram is larger than the MTU of the next link, the router splits it. Each fragment is a complete IP datagram with:

- the same **identification** number,
- the **MF** flag set on all but the last,
- a **fragment offset** in units of 8 bytes,

- a recalculated header checksum, and
- the data payload length adjusted, in multiples of 8.

**Exam Focus**

**Worked.** A 4000-byte datagram (20-byte header, 3980 bytes of data) must cross a link with MTU 1500. How many fragments?

**Solution**

Each fragment carries at most $1500 - 20 = 1480$ bytes of data, and that must be a multiple of 8. $1480/8 = 185$, so 1480 is legal.

$$\lceil 3980/1480 \rceil = 3 \text{ fragments}$$

| Fragment | Data bytes | Offset (8-byte units) | MF |
|---|---|---|---|
| 1 | 1480 | 0 | 1 |
| 2 | 1480 | 185 | 1 |
| 3 | 1020 | 370 | 0 |

Total data $= 1480 + 1480 + 1020 = 3980$, which reconstructs exactly. Offsets accumulate in units of 8 bytes, so 185 and 370, not 1480 and 2960.

## ICMP

**Past-paper question**

**Oct 2025 Q2(a)(i).** Explain how ICMP works in network troubleshooting. Illustrate with examples of echo request and destination unreachable messages.

**Solution**

**ICMP (Internet Control Message Protocol)** is IP's error-reporting and diagnostic companion. It is carried *inside* IP datagrams (protocol number 1) and is used by routers and hosts to report problems; it does not make IP reliable, because an ICMP message is never sent in response to another ICMP message.

| **Message** | Meaning and use |
|---|---|
| **Echo request / reply (type 8 / 0)** | `ping`. Confirms reachability and measures round-trip time. |
| **Destination unreachable (type 3)** | The network, host, protocol or port cannot be reached, or fragmentation was needed but DF was set. Code 3 (port unreachable) is how `traceroute` discovers a hop. |
| **Time exceeded (type 11)** | TTL reached zero. This is what `traceroute` exploits. |
| **Redirect (type 5)** | A better first-hop router exists. |
| **Source quench** | Obsolete congestion signal; deprecated. |

**Worked diagnostic sequence.** A user cannot reach a server.

1. `ping 127.0.0.1` — is the local stack working?
2. `ping <own IP>` — is the interface configured?
3. `ping <default gateway>` — is the local link and gateway reachable? An echo reply proves layer 3 to the gateway.
4. `ping <server IP>` — is the remote host reachable? If this works but the name does not, the problem is DNS.
5. `traceroute <server>` — where do the Time Exceeded replies stop? That hop is the fault.

**Example.** `ping` to a host returns *Destination Host Unreachable* from the local router. That means the router has no route to the destination network, or ARP for the destination failed on the final link. Either way the

fault is at or beyond the router, not on the local host.

## ARP

**Past-paper question**

**Oct 2024 CAE-II Q2A.** Explain how ARP can be used to resolve communication failure between two devices due to incorrect MAC address mapping in a network.

**Solution**

**ARP (Address Resolution Protocol)** maps a known IPv4 address to the unknown MAC address on the same LAN. IP routing needs a MAC address to build the frame, but the application only knows the IP.

**The exchange:**

1. A wants to send to B on the same subnet, but has only B's IP.
2. A broadcasts an **ARP request**: "Who has 192.168.1.20? Tell 192.168.1.10."
3. Every host on the LAN receives it; only B replies, **unicast**, with its MAC.
4. A caches the mapping in its ARP table for a few minutes.

**If A and B are on different subnets**, A ARPs for the *default gateway*, not for B.

**Troubleshooting an incorrect MAC mapping.** Symptoms: `ping` fails with *Destination Host Unreachable* even though the IP configuration looks right; the ARP table shows an entry that does not match the device's real NIC.

- Inspect the table with `arp -a` or `ip neigh`.
- A stale entry after a NIC replacement or an IP move is the common cause: clear it with `arp -d` and let ARP re-resolve.
- If the entry keeps returning to the wrong MAC, suspect **ARP spoofing** (a host answering for an address it does not own). Defences: static ARP entries for critical hosts, Dynamic ARP Inspection on the switch, port security.
- If the table stays empty, the failure is at layer 2: VLAN mismatch, a blocked port, or the two hosts are not actually in the same broadcast domain.

**Gratuitous ARP** is an unsolicited ARP that announces "this IP is now at this MAC"; it is how a host updates everyone after an address change or a failover.

## IPv6

**Past-paper question**

**Oct 2025 Q2(a)(iii).** Explain how IPv6 simplifies packet processing compared to IPv4.

**Solution**

| Feature | IPv4 | IPv6 |
|---|---|---|
| **Address size** | 32 bits | 128 bits |
| **Header** | 20–60 bytes, 13 fields | Fixed 40 bytes, 8 fields |
| **Checksum** | Header checksum, recomputed at each hop | **None**; left to link and transport layers |
| **Fragmentation** | Routers may fragment | **Sources only**; routers never fragment |
| **Options** | In the base header, variable length | Moved to **extension headers** |
| **Broadcast** | Yes | **No**; replaced by multicast and anycast |
| **Address configuration** | Manual or DHCP | **SLAAC**: stateless autoconfiguration |
| **Security** | IPSec optional, bolt-on | IPSec mandatory in the standard |

**Why IPv6 is simpler to process:**

1. **Fixed 40-byte header** — no IHL field to decode, and hardware can parse at a fixed offset.
2. **No header checksum** — routers no longer recompute a checksum at every hop, which was a significant per-packet cost.
3. **No router fragmentation** — routers just drop and send ICMP Packet Too Big; the source adapts. Less state and work per packet.
4. **Extension headers** — optional features are chained after the fixed header, so a router that does not need them skips them entirely.
5. **Hierarchical addressing with aggregation** — smaller routing tables in the default-free zone.

**Notation.** Eight groups of four hex digits, `2001:0db8:0000:0000:0000:ff00:0042:8329`, compressible to `2001:db8::ff00:42:8329`: drop leading zeros in each group, and replace one run of all-zero groups with `::` (only once).

# Routing algorithms

## The three families

| Family | Knowledge required | Example and note |
|---|---|---|
| **Distance vector** | Only its own neighbours | RIP. Each router tells its neighbours its whole distance table. "Routing by rumour." |
| **Link state** | The whole topology | OSPF, IS-IS. Each router floods link-state advertisements and runs Dijkstra locally. |
| **Path vector** | The whole path, per destination | BGP. Carries the AS path, which is used for loop detection and policy. |

## Link state: Dijkstra

**Past-paper question**

**Worked, standard form.** Find the shortest path tree from A.

| Link | Cost |
|---|---|
| A–B | 2 |
| A–C | 5 |
| B–C | 1 |
| B–D | 4 |
| C–D | 2 |
| C–E | 6 |

**Solution**

Dijkstra settles one node per step, always the unsettled node with the smallest tentative distance.

| Step | Settled | Tentative distances | Chosen |
|---|---|---|---|
| 0 | — | A=0, others $\infty$ | A |
| 1 | A | B=2, C=5 | B (2) |
| 2 | B | C=$\min(5,\ 2+1) = 3$, D=$2+4 = 6$ | C (3) |
| 3 | C | D=$\min(6,\ 3+2) = 5$, E=$3+6 = 9$ | D (5) |
| 4 | D | E=$\min(9,\ 5+1) = 6$ | E (6) |
| 5 | E | — | done |

**Result:**

$$A = 0, \quad B = 2, \quad C = 3, \quad D = 5, \quad E = 6$$

**Shortest paths:** B via A–B; C via A–B–C; D via A–B–C–D; E via A–B–C–D–E.

**Do Not Confuse**

**The mistake everyone makes:** updating the distance but not the path. D's distance of 5 arrives via **C**, not directly from B, even though B is D's neighbour with cost 4. The route is A–B–C–D. Write the predecessor down as you go.

**Complexity:** $O(n^2)$ with a simple array, $O((n+e)\log n)$ with a heap. Because every router computes the whole tree, link state costs more CPU but converges faster than distance vector.

### Distance vector: Bellman-Ford

Each router keeps a vector of distances to every destination and periodically sends it to its neighbours. On receipt, for each destination $X$:

$$D(v,X) = \min_{w \in \text{neighbours}(v)} \big[c(v,w) + D(w,X)\big].$$

This is the Bellman-Ford equation. Running it to convergence on the network above gives the same distances as Dijkstra: A=0, B=2, C=3, D=5, E=6, after two rounds.

**Do Not Confuse**

**Count-to-infinity.** Distance vector's weakness. If a link fails, two routers can each believe the other has a route, and the reported distance climbs slowly to infinity instead of failing fast. Fixes: a **definition of infinity** (RIP uses 16), **split horizon** (do not advertise a route back the way it came), **poison reverse**, and **triggered updates**. If a question asks "why does RIP converge slowly", this is the answer.

### RIP versus OSPF

**Past-paper question**

**Sept 2026 CAE-1 Q5A.** Explain the working of RIP and OSPF. Compare RIP and OSPF based on routing algorithm, metric, convergence, scalability, and suitability for different network sizes.

**Solution**

**RIP** (Routing Information Protocol) is a distance-vector protocol. Each router advertises its full table to its neighbours every 30 seconds; the metric is **hop count**, with a maximum of 15 and 16 meaning unreachable. It is simple to configure and needs no topology knowledge, but it converges slowly, scales poorly, and ignores link speed entirely — a route over three slow serial links looks identical to three fast ones.

**OSPF** (Open Shortest Path First) is a link-state protocol. Each router floods link-state advertisements describing its own links and costs; every router builds an identical topology database and runs Dijkstra. The metric is **cost**, derived from bandwidth ($10^8$/bandwidth by default). It converges quickly using triggered updates and flooding, supports areas to bound the flood, and authenticates its updates.

| **Basis** | RIP | OSPF |
|---|---|---|
| **Algorithm** | Distance vector (Bellman-Ford) | Link state (Dijkstra) |
| **Metric** | Hop count, max 15 | Cost based on bandwidth |
| **Convergence** | Slow, minutes; count-to-infinity | Fast, seconds; triggered updates |
| **Scalability** | Poor: small networks only | Good: hierarchical areas |
| **Updates** | Full table every 30 s to neighbours | Triggered LSAs flooded to the area |
| **Bandwidth awareness** | None | Yes |
| **CPU and memory** | Very low | Higher: full topology database |
| **Suitability** | Small, simple networks | Large enterprise and ISP networks |

**One-line answer if asked to choose:** RIP for a handful of routers where simplicity matters; OSPF as soon as the network has more than about fifteen hops, redundant paths, or a need to converge quickly.

## Path vector and BGP

**Past-paper question**

**Oct 2026 CAE-II Q2A.** Analyse how Autonomous Systems exchange routing information and how BGP supports communication between different networks on the Internet.

**Solution**

**BGP (Border Gateway Protocol)** is the path-vector protocol that glues the Internet together. It runs between routers in *different* Autonomous Systems (eBGP) and within one AS (iBGP).

**Why distance vector is not enough between ISPs:** an ISP must be able to enforce *policy* ("do not carry traffic for that competitor"), and it must detect loops over a graph of thousands of networks. Counting hops does not express policy.

**How BGP works:**

1. An AS advertises the **prefixes** it owns along with the **AS path** — the sequence of AS numbers the advertisement traversed.
2. A router receiving an advertisement prepends its own AS number before passing it on.
3. A router that finds its own AS number in the path **rejects** the route: that is loop detection, for free.
4. Among the feasible routes, the router selects one using a decision process: highest local preference, shortest AS path, lowest origin type, lowest MED, then eBGP over iBGP, then lowest IGP cost to the next hop, then lowest router ID.
5. Only the chosen route is installed in the forwarding table, but all are kept in the BGP table.

**Consequences.** BGP converges slowly (minutes) and its correctness depends on policy rather than on a metric, which is why a misconfiguration at one large ISP can take down a large part of the Internet. That fragility is the price of policy control.

## README: routing tables

**Past-paper question**

**Oct 2025 Q2(b)(ii).** Analyse the need for routing tables and how they are updated in dynamic routing.

**Solution**

A router must decide, for every packet, which interface to use. The **routing table** holds the mapping from destination prefix to next hop and interface — it *is* the router's decision data.

**Static routing** fills the table by hand. It is predictable and has no protocol overhead, but it cannot react to a link failure and does not scale.

**Dynamic routing** builds and maintains the table automatically:

- protocols exchange topology or distance information,
- the best path per prefix is computed (Dijkstra for link state, Bellman-Ford for distance vector),
- the result is installed in the routing table,
- on a topology change, an update is flooded and the table reconverges.

**Longest-prefix match.** If several entries match, the most specific wins: a packet to `10.1.2.3` uses `10.1.2.0/24` rather than `10.0.0.0/8`, whatever the metric. Then administrative distance decides between sources (connected beats static beats OSPF beats RIP), then the metric.

## Answer skeletons

**Answer Skeleton**

**Subnetting (10 marks).** Write the prefix, the borrowed bits, $h = 32 - n$, block size $= 2^h$, usable $= 2^h - 2$. Draw the table: subnet number, network, first host, last host, broadcast. Check that each broadcast is one below the next network.

**Answer Skeleton**

**Shortest path (10 marks).** Draw the graph with costs on the edges. Draw the step table with settled nodes and tentative distances. State the final distances and the paths, not just the distances.

**Answer Skeleton**

**Protocol comparison (10 marks).** A table with at least five rows: algorithm, metric, convergence, scalability, suitability. Then one sentence saying which you would choose and why.

**Answer Skeleton**

**Datagram structure (10 marks).** Draw the header as a row of boxes to scale, label each field with its bit width, then list the function of each field below. Twelve fields named correctly is most of the marks.

## Self-test

1. List six functions of the network layer.
2. Why is the network layer described as connectionless and best-effort?
3. Differentiate routing from forwarding.
4. State the class, default mask and first-octet range for classes A, B and C.
5. Subnet 192.168.10.0/24 into 8 subnets. Give the prefix, block size and usable hosts.
6. A network needs 50 hosts. What prefix and how many usable addresses?
7. Summarise 200.1.0.0/24 through 200.1.3.0/24 into one prefix.
8. Name ten fields of the IPv4 header with their sizes.
9. Why is there no header checksum in IPv6?
10. A 4000-byte datagram crosses an MTU of 1500. How many fragments and what are the offsets?
11. Why does IPv6 forbid router fragmentation?
12. Explain ARP in four steps.
13. How does ARP help diagnose a duplicate-IP or spoofing problem?
14. What does ICMP Time Exceeded do for traceroute?
15. Compare RIP and OSPF in five rows.
16. State the Bellman-Ford equation.
17. What is count-to-infinity and how is it mitigated?
18. Why is BGP a path-vector protocol rather than a distance-vector one?
19. What is longest-prefix match?

### Answers

1. Logical addressing, routing, forwarding, packetising, fragmentation and reassembly, error reporting, QoS marking, NAT.
2. It offers no guarantees of delivery, ordering or duplicate suppression, so IP can run over any link and TCP can add reliability where needed.
3. Routing builds the table periodically from protocol information. Forwarding moves one packet using the table, per packet.
4. A: 1–126, /8. B: 128–191, /16. C: 192–223, /24.
5. /27, block 32, usable 30.
6. $2^h - 2 \geq 50$, so $h = 6$, prefix /26, 62 usable.

7. /22, that is 200.1.0.0/22.
8. Version, IHL, DSCP, total length, identification, flags, fragment offset, TTL, protocol, header checksum, source, destination.
9. Because a transport or link checksum already covers the data, the per-hop recomputation was redundant and expensive.
10. Three fragments: data 1480, 1480 and 1020; offsets 0, 185 and 370.
11. To keep routers stateless and fast; the source learns the path MTU from ICMP and fragments itself.
12. Broadcast request, unicast reply, cache the mapping, refresh on timeout.
13. A persistent wrong MAC for a known IP indicates spoofing; static entries, DAI and port security defend against it.
14. Each router sets TTL to 1, 2, 3, so each hop in turn returns Time Exceeded, revealing the path.
15. See the table: distance vector versus link state, hops versus cost, slow versus fast, small versus large.
16. $D(v, X) = \min_w[c(v, w) + D(w, X)]$.
17. A failed link is masked because neighbours keep advertising a route learnt through each other; fixed by infinity limits, split horizon, poison reverse and triggered updates.
18. ISPs need policy control and loop detection over a graph of thousands of ASes; the AS path supplies both.
19. When several prefixes match, the most specific (longest) prefix is used.

## Last-minute page

**NETWORK LAYER**
Logical addressing, routing,
forwarding, fragmentation,
ICMP, NAT
Connectionless, best effort

**SUBNET MATHS**
$h = 32 - n$
block $= 2^h$
usable $= 2^h - 2$
/24: 256, 254 hosts
/25: 128, 126
/26: 64, 62
/27: 32, 30
/28: 16, 14

**AGGREGATION**
$2^k$ prefixes $\rightarrow$ subtract $k$

**IPv4 HEADER**
version, IHL, DSCP,
total length, ID, flags,
offset, TTL, protocol,
checksum, src, dst

**FRAGMENT OFFSET**
in 8-byte units

**ICMP**
echo 8/0, unreachable 3,
time exceeded 11

**ARP**
broadcast request,
unicast reply, cache

**ROUTING**
DV: neighbours, RIP,
hops, max 15
LS: flood, OSPF,
Dijkstra, cost
PV: BGP, AS path

**TRAPS**
RIP max hop = 15, 16 = unreachable
Dijkstra: track the predecessor
Fragments in 8-byte units
IPv6 has no checksum

*For any subnetting question, write the block size first. Everything else follows from it.*